Private Telegram assistant

Two components. One private chat. A bounded agent loop with local MCP tools.

Target architecture

Telegram edge: receive updates, verify webhook secret and user ID, accept private chats, normalize messages, send typing actions, call the backend with native fetch, and deliver replies.

FastAPI container: authenticate edge requests, load chat context, start allowlisted MCP subprocesses, expose approved tool schemas to one model provider, run tool calls, persist the result, and return structured JSON.

Important runtime distinction: Telegram supplies the Bot API. Your chosen edge host supplies the isolate and its limits. Treat package installation and subprocess execution as unavailable at the edge, then verify the actual host limits before deployment.

Request and response flow

1. Telegram sends an update to the edge webhook.
2. Edge rejects unauthorized senders before backend work.
3. Edge sends a typing action and POSTs a normalized request.
4. Backend connects to MCP, lists approved tools, and starts the model loop.
5. Model requests tools. Backend validates arguments and executes them.
6. Backend returns final text. Edge splits and sends the answer.

MVP boundary

Start with text, one allowed Telegram user, one model provider, and one read-only MCP server. Add bounded chat history after tool execution works. Add voice and write actions later.

Synchronous execution decision

Keep the direct POST flow only for runs finishing inside every edge, proxy, webhook, and container request limit, including cold starts. Set one total deadline below the smallest measured limit. If the deadline expires, stop the run and return a short failure. Do not launch detached work after returning a response.

First deployment gate

Confirm the container permits subprocesses, outbound model/tool traffic, and the required Python or Node runtime. Install and pin MCP packages at image build time. Measure a cold-start tool run before selecting synchronous delivery.

Build the interface

Milestone 1: secure edge and a working backend stub.

01. Create the project structure

Use edge/handler.ts and edge/config.ts. Use backend/app/main.py, schemas.py, auth.py, agent.py, mcp_manager.py, storage.py, and providers/. Add backend/tests/, Dockerfile, a dependency lockfile, and .env.example.

Complete when: Both components start independently. No secrets enter source control.

02. Protect the Telegram webhook

Register HTTPS webhook with a secret_token. Validate X-Telegram-Bot-Api-Secret-Token. Check message.from.id against ALLOWED_USER_ID and require private chat type. Handle missing message fields safely.

Complete when: Unauthorized updates trigger no model calls. Edited messages and unsupported update types follow a documented ignore policy.

03. Normalize input and deduplicate updates

Extract text or caption, update_id, chat_id, message_id, and sender ID. Set a prompt length limit. Use update_id as the request deduplication key. Put authoritative deduplication in shared backend storage, not isolate memory.

Complete when: Replaying the same update reuses its job/result and never repeats a tool mutation.

04. Connect edge to FastAPI

POST /v1/assistant/run with JSON and an Authorization bearer secret. Use fetch with an abort deadline. Send typing before the call and refresh only while the host keeps the request alive.

Complete when: A backend stub returns text to the same private chat. Auth failures, timeout, and unavailable backend produce concise replies.

05. Deliver Telegram-safe text

Use plain text initially. Split final answers into ordered chunks below the 4096-character sendMessage limit, preserving Unicode boundaries. Record returned Telegram message IDs. Handle 429 using retry_after within the delivery budget.

Complete when: Long answers arrive in order without formatting errors. Delivery failures remain distinct from agent failures.

HTTP contract

```
POST /v1/assistant/run
Authorization: Bearer <edge-backend-secret>
{request_id: "tg:12345", update_id: 12345, user_id: 123,
chat_id: 123, message_id: 88, text: "Summarize my notes"}

{request_id: "tg:12345", status: "completed",
answer: "...", error_code: null}
```

Use actual valid JSON in implementation. Validate every field with Pydantic. Return 401 for bad backend credentials, 403 for disallowed identity, 422 for invalid input, and a stable structured error for deadline or provider failures.

Build the execution engine

Milestone 2: one real MCP tool runs through the model.

06. Implement MCP subprocess lifecycle

Use the official MCP SDK client and stdio transport. Load server command, fixed arguments, permitted environment variables, and tool allowlist from trusted configuration. Initialize sessions and discover tools. Namespace tool names to avoid collisions.

Complete when: A tool executes successfully. Failure, timeout, cancellation, and normal completion all close sessions and reap child processes.

07. Implement the model adapter

Choose Claude or GPT first. Convert MCP tool schemas into the selected provider format. Preserve provider tool-call IDs and return results in the required format. Keep model-specific code behind one adapter.

Complete when: A fake model test and one live smoke run both request a tool, consume its result, and produce final text.

08. Implement the bounded agent loop

Send system policy, bounded conversation history, and user input. Execute validated approved tool calls, append tool results, and request the next model turn. Stop on final answer, deadline, cancellation, or limits.

Complete when: A multi-step read task finishes. Limit exhaustion returns a useful partial-status message without claiming success.

Suggested initial limits, tune after measurement

Maximum 6 model turns, 8 total tool calls, and 1 active run per chat. Bound each tool result and the combined context size. Derive total run time from the host limits. Keep per-tool timeout shorter than the remaining total deadline.

09. Add tool permission enforcement

Begin with read-only tools. Never let model output select arbitrary executable commands, environment variables, filesystem roots, or callback URLs. Treat forwarded content and tool output as untrusted input. Enforce access checks in code before each call.

Complete when: A prompt attempting to bypass tool permissions fails before execution.

10. Add context and durable request state

Store request ID, update ID, user/chat IDs, status, timestamps, bounded history, answer, and error code. Serialize runs per chat. Use a shared store for multiple instances. Support /reset to clear chat context.

Complete when: Container restart preserves completed requests. Duplicate input and simultaneous chat messages produce predictable results.

Write actions, only after the read-only MVP

Represent a proposed action as a stored record with validated arguments and a short-lived confirmation token. Display the exact action in Telegram. Verify sender, action ID, expiry, and unused state on confirmation. Execute once using an idempotency key. A model sentence expressing approval is insufficient.

Add media and long runs

Milestone 3: voice support and reliable delivery beyond request limits.

11. Handle voice notes

Edge forwards Telegram file_id and metadata. Backend fetches the file through Telegram using a backend-held bot token, enforces size/type limits, and sends it to speech transcription. Keep download URLs and tokens out of logs. Pass transcript through the same text pipeline.

Complete when: A voice note produces a transcript-based answer. Oversized, unsupported, and failed downloads return controlled errors.

12. Handle forwards and captions

Treat forwarded text as user-provided content. Keep attribution as optional metadata, never as sender authorization. Support text and media captions first. Add document/image processing only through separate explicit handlers.

Complete when: Forwarded instructions never change access policy. Unsupported media receives a clear response.

Durable long-run mode

Use this mode once measured execution exceeds the synchronous budget. It preserves the two-component ownership model, with a managed queue and shared storage supporting the backend.

1. Edge calls POST /v1/assistant/jobs.
2. Backend durably records and queues the job, then returns 202 with job_id.
3. Edge acknowledges the Telegram webhook and sends a short receipt.
4. A queue-triggered container runs the same MCP and model engine.
5. Backend POSTs a signed completion event to a dedicated edge endpoint.
6. Edge calls sendMessage and records delivery state.

13. Implement queued execution and callback delivery

Use a provider-supported durable job queue, not in-process FastAPI BackgroundTasks for required completion. Authenticate callbacks with a timestamped HMAC over the body. Validate expiry, event ID, and job identity. Persist callback/delivery state and retry failed delivery with bounded backoff.

Complete when: A worker restart resumes or safely retries a job. Repeated completion events do not normally repeat replies.

Delivery limitation to document

Telegram sendMessage has no application idempotency key. If delivery succeeds but the acknowledgement is lost, a retry risks a duplicate. Keep delivery state and message IDs, separate tool execution from delivery retries, and never rerun tools merely to resend an answer.

14. Add cancellation and recovery

Implement /cancel for the current chat job. Store cancellation state and check it before each model/tool step. Stop active work where supported, close MCP sessions, and record cancelled status. Retry transient provider failures only inside the budget.

Complete when: Cancellation prevents the next tool call. Completed write actions remain recorded and are never described as rolled back without evidence.

Ship and verify

Milestone 4: deploy a small private assistant with clear operating limits.

Configuration checklist

Edge: TELEGRAM_BOT_TOKEN, TELEGRAM_WEBHOOK_SECRET, ALLOWED_USER_ID, BACKEND_URL, EDGE_BACKEND_SECRET, and optional CALLBACK_SIGNING_SECRET.
Backend: selected model API key and model ID, EDGE_BACKEND_SECRET, ALLOWED_USER_ID, MCP server config, MCP-specific credentials, shared storage URL, run/tool limits, and optional queue/callback settings. Backend also needs TELEGRAM_BOT_TOKEN for voice downloads.

Deployment tasks

15. Build a pinned container image with MCP dependencies preinstalled. Run as a non-root user. Pass only required secrets to each MCP process.
16. Deploy backend and verify health/readiness plus an authenticated test request. Set concurrency and per-chat locking.
17. Deploy edge, register webhook, then test with the allowed Telegram account.
18. Add structured logs with request/job IDs, durations, tool names, status, and model usage. Redact tokens, media URLs, personal content, and tool results by default.
19. Set retention for history, audio, and job data. Delete temporary media after processing. Add a usage cap and a simple failure alert.

Required acceptance tests

A. Unauthorized sender and invalid webhook secret never reach the model.
B. One text request and one two-tool task finish end to end.
C. Duplicate updates never repeat a tool mutation.
D. MCP crash, invalid arguments, oversized result, and provider failure return safe errors.
E. Deadline and cancellation leave no orphan subprocesses.
F. Two messages from one chat execute in order.
G. A long answer splits correctly. A voice note follows the text pipeline.
H. Cold start fits the selected mode. Queued jobs survive worker interruption.
I. Forged/replayed callbacks fail. Delivery retries never rerun the agent.
J. Write confirmations reject wrong users, expired tokens, and repeated execution.

Coder handoff and completion order

Deliver source, lockfiles, Dockerfile, .env.example, API schemas, automated tests, and a README with local setup, webhook registration, deployment, and recovery steps. Complete tasks 01-09 first. Add state and media next. Enable long-run mode before accepting tasks exceeding measured request limits.

Reference documentation

```
Telegram Bot API: https://core.telegram.org/bots/api
MCP Python SDK: https://py.sdk.modelcontextprotocol.io/run/
FastAPI background-task caveat: https://fastapi.tiangolo.com/tutorial/background-tasks/#caveat
```

Host-specific limits remain a deployment decision because the edge and container providers were not specified. Verify request duration, background execution, queue support, and subprocess permissions against the selected providers.